

Queue (Antrian)

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung



He thinks he has to wait in line to get a treat.

Queue



Queue adalah sederetan elemen yang:

- dikenali elemen pertama (HEAD) dan elemen terakhirnya (TAIL).
- aturan penyisipan dan penghapusan elemennya didefinisikan sebagai berikut:
Penyisipan selalu dilakukan setelah **elemen terakhir**,
Penghapusan selalu dilakukan pada **elemen pertama**.

Queue

Elemen Queue tersusun secara **FIFO** (*First In First Out*)

Contoh pemakaian Queue:

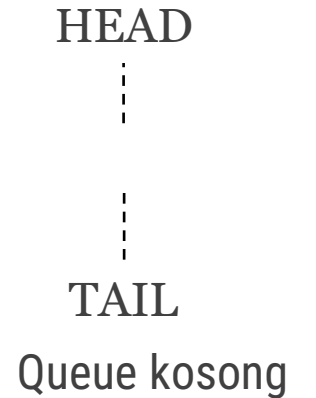
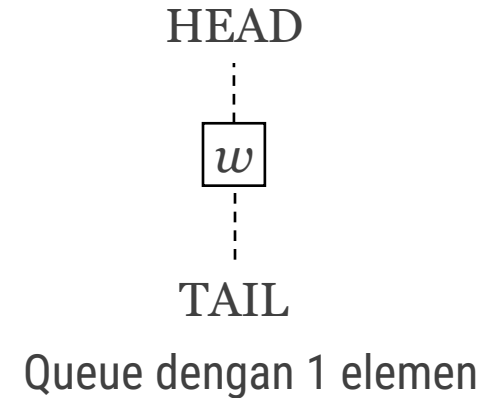
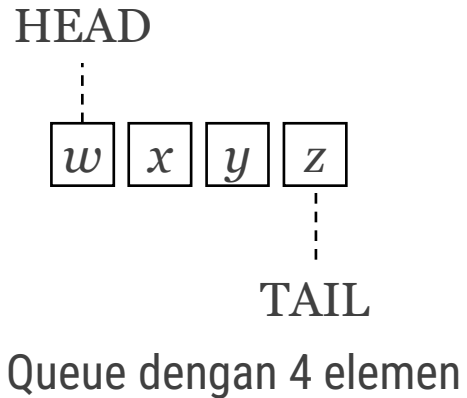
- antrian job yang harus ditangani oleh sistem operasi (*job scheduling*).
- antrian dalam dunia nyata.

Queue seperti sebuah List dengan batasan lokasi penambahan & pengurangan elemen.

Queue

Secara logik:

- Elemen
- Head (elemen terdepan)
- Posisi tail (elemen paling belakang)
- Queue kosong



Definisi operasi

Jika diberikan Q adalah Queue dengan elemen $ElmtQ$

$CreateQueue: \rightarrow Q$	{ Membuat sebuah antrian kosong }
$head: Q \rightarrow ElmtQ$	{ Mengirimkan elemen terdepan Q saat ini }
$length: Q \rightarrow \underline{integer}$	{ Mengirimkan banyaknya elemen Q saat ini }
$enqueue: ElmtQ \times Q \rightarrow Q$	{ Menambahkankan sebuah elemen setelah elemen paling belakang Queue }
$dequeue: Q \rightarrow Q \times ElmtQ$	{ Menghapus kepala Queue, mungkin Q menjadi kosong }
$isEmpty: Q \rightarrow \underline{boolean}$	{ Tes terhadap Q : true jika Q kosong, false jika Q tidak kosong }

Axiomatic semantics (fungsional)

- 1) `new()` returns a queue
- 2) `head(enqueue(v, new())) = v`
- 3) `dequeue(enqueue(v, new())) = new()`
- 4) `head(enqueue(v, enqueue(w, Q))) = head(enqueue(w, Q))`
- 5) `dequeue(add(v, enqueue(w, Q))) = add(v, dequeue(enqueue(w, Q)))`

Di mana Q adalah Queue dan v, w adalah value.

Implementasi Queue dengan array

Memori tempat penyimpan elemen adalah sebuah array dengan indeks $0..CAPACITY-1$.

Perlu informasi indeks array yang menyatakan posisi Head dan Tail.

ADT Queue dengan array

KAMUS UMUM

constant IDX_UNDEF: integer = -1

constant CAPACITY: integer = 10

type ElType: integer { *elemen Queue* }

{ *Queue dengan array statik* }

type Queue: < buffer: array [0..CAPACITY-1] of ElType, { *penyimpanan elemen* }
 idxHead: integer, { *indeks elemen terdepan* }
 idxTail: integer > { *indeks elemen terakhir* }

ADT Queue – Konstruktor, akses, & predikat

procedure CreateQueue(output q: Queue)

{ I.S. Sembarang

*F.S. Membuat sebuah Queue q yang kosong berkapasitas CAPACITY
jadi indeksnya antara 0..CAPACITY-1*

Ciri Queue kosong: idxHead dan idxTail bernilai IDX_UNDEF }

function head(q: Queue) → ElType

{ Prekondisi: q tidak kosong.

Mengirim elemen terdepan q, yaitu q.buffer[q.idxHead]. }

function length(q: Queue) → integer

{ Mengirim jumlah elemen q saat ini }

function isEmpty(q: Queue) → boolean

{ Mengirim true jika q kosong: lihat definisi di atas }

function isFull(q: Queue) → boolean

{ Mengirim true jika penyimpanan q penuh }

ADT Queue - Operasi

procedure enqueue (input/output q: Queue, input val: ElType)

{ Menambahkan val sebagai elemen Queue q.

I.S. q mungkin kosong, TIDAK penuh

F.S. q bertambah elemen val sebagai tail yang baru }

procedure dequeue (input/output q: Queue, output val: ElType)

{ Menghapus head dari Queue q.

I.S. q tidak kosong

F.S. val berisi nilai head yang lama.

Jika q tidak menjadi kosong,

q.idxHead berpindah ke elemen berikutnya pada q.

Jika q menjadi kosong,

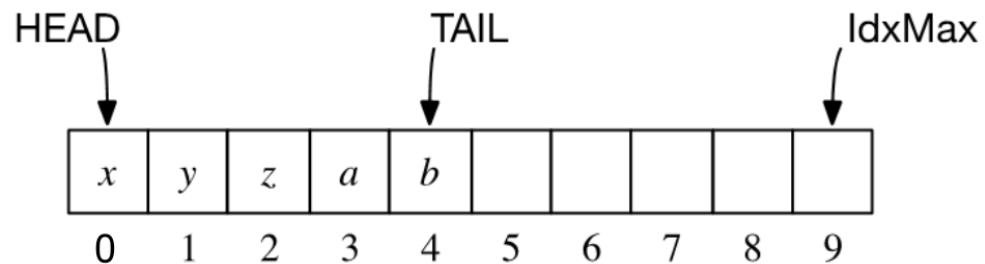
q.idxHead dan q.idxTail menjadi bernilai IDX_UNDEF. }

Implementasi Queue dengan array – alt-1

Jika Queue tidak kosong: `idxTail` adalah indeks elemen terakhir, `idxHead` selalu diset = 0.

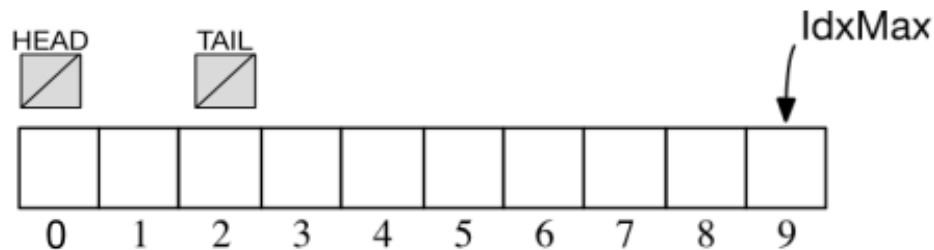
Jika Queue kosong, maka `idxHead` dan `idxTail` diset = `IDX_UNDEF`.

- Ilustrasi Queue tidak kosong dengan 5 elemen:



*dengan `IdxMax` = `CAPACITY-1`

- Ilustrasi Queue kosong:



Implementasi Queue dengan array – alt-1

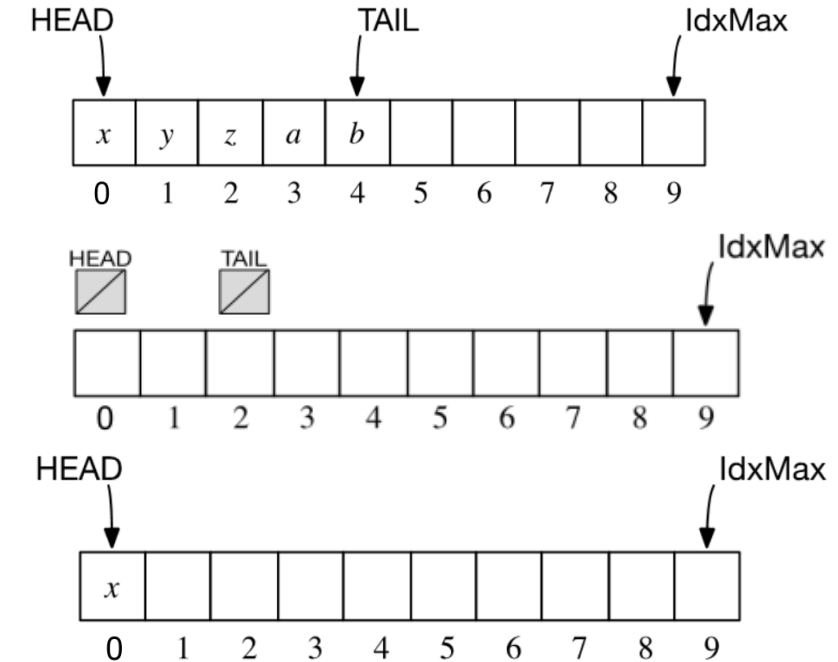
Algoritma **penambahan elemen**:

- **Jika masih ada tempat**: geser TAIL ke kanan.
- **Kasus khusus** (Queue kosong): **idxHead** dan **idxTail** diset = 0.

Algoritma paling sederhana dan “naif” untuk **penghapusan elemen**:

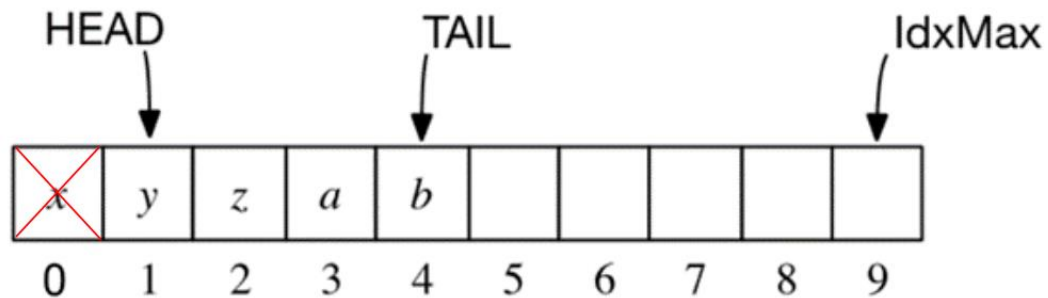
- **Jika Queue tidak kosong**: ambil nilai elemen HEAD, geser semua elemen mulai dari **idxHead+1** s.d. **idxTail**, kemudian geser TAIL ke kiri.
- **Kasus khusus** (Queue berelemen 1): **idxHead** dan **idxTail** diset = **IDX_UNDEF**.

Algoritma ini mencerminkan pergeseran orang yang sedang mengantri di dunia nyata, tapi tidak efisien.

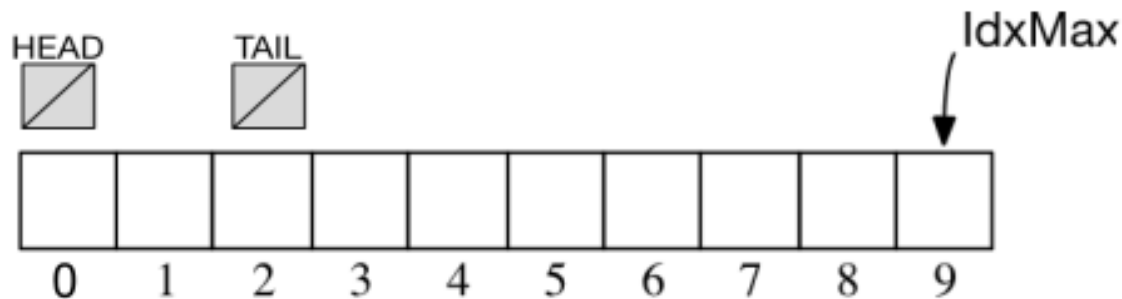


Implementasi Queue dengan array – alt-2

Tabel dengan representasi HEAD dan TAIL yang mana **HEAD bergeser ke kanan** ketika sebuah elemen dihapus.

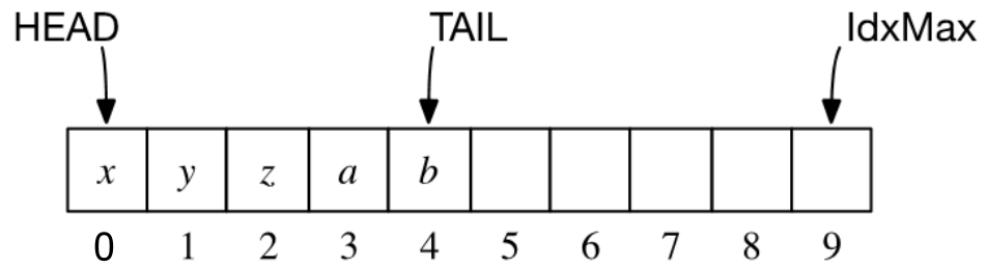


Jika Queue kosong, maka `idxHead` dan `idxTail` diset = `IDX_UNDEF`.

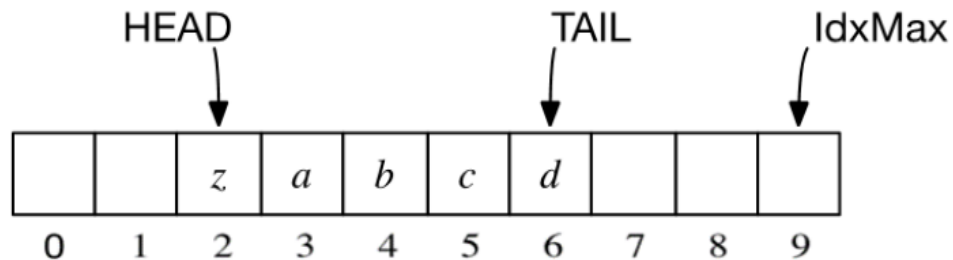


Implementasi Queue dengan array – alt-2

Ilustrasi Queue tidak kosong, dengan 5 elemen, kemungkinan pertama HEAD sedang berada di indeks 0:



Ilustrasi Queue tidak kosong, dengan 5 elemen, kemungkinan lain HEAD tidak berada di indeks 0 (akibat algoritma penghapusan):



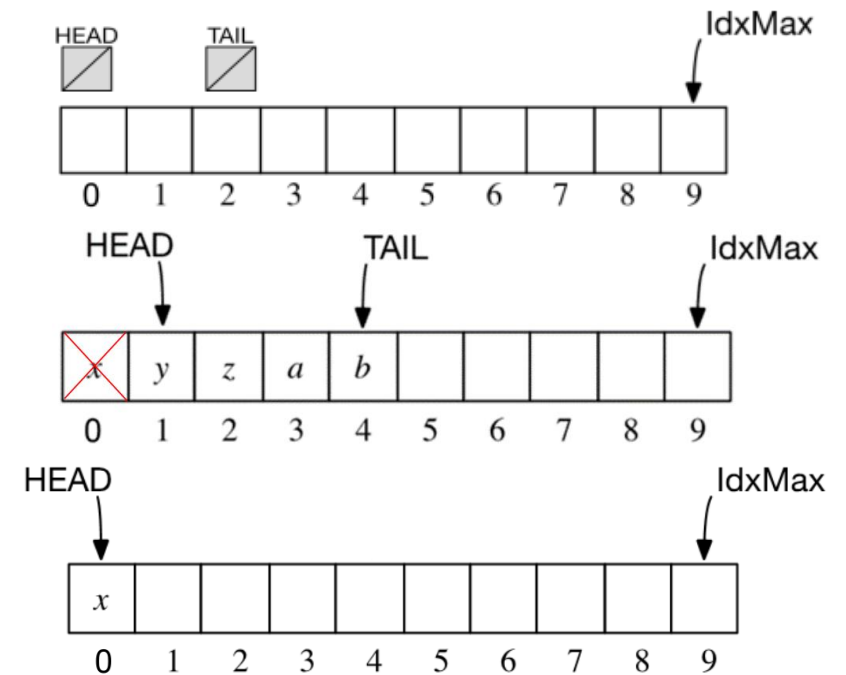
Implementasi Queue dengan array – alt-2

Algoritma **penambahan elemen** sama dengan alt-1, kecuali pada saat “penuh semu” (lihat slide berikutnya.)

Algoritma **penghapusan elemen**:

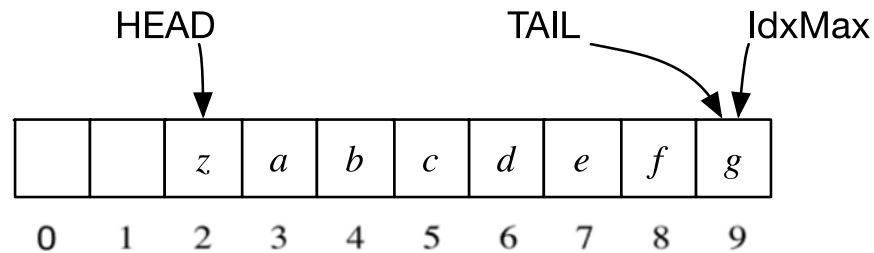
- **Jika Queue tidak kosong:** ambil nilai elemen HEAD, kemudian HEAD digeser ke kanan.
- **Kasus khusus (Queue berelemen 1):** `idxHead` dan `idxTail` diset = `IDX_UNDEF`.

Algoritma ini **TIDAK** mencerminkan pergeseran orang yang sedang mengantri di dunia nyata, tapi **efisien**.



Implementasi Queue dengan array – alt-2

Keadaan Queue penuh tetapi “semu” sebagai berikut:



Harus dilakukan aksi menggeser elemen untuk menciptakan ruangan kosong.

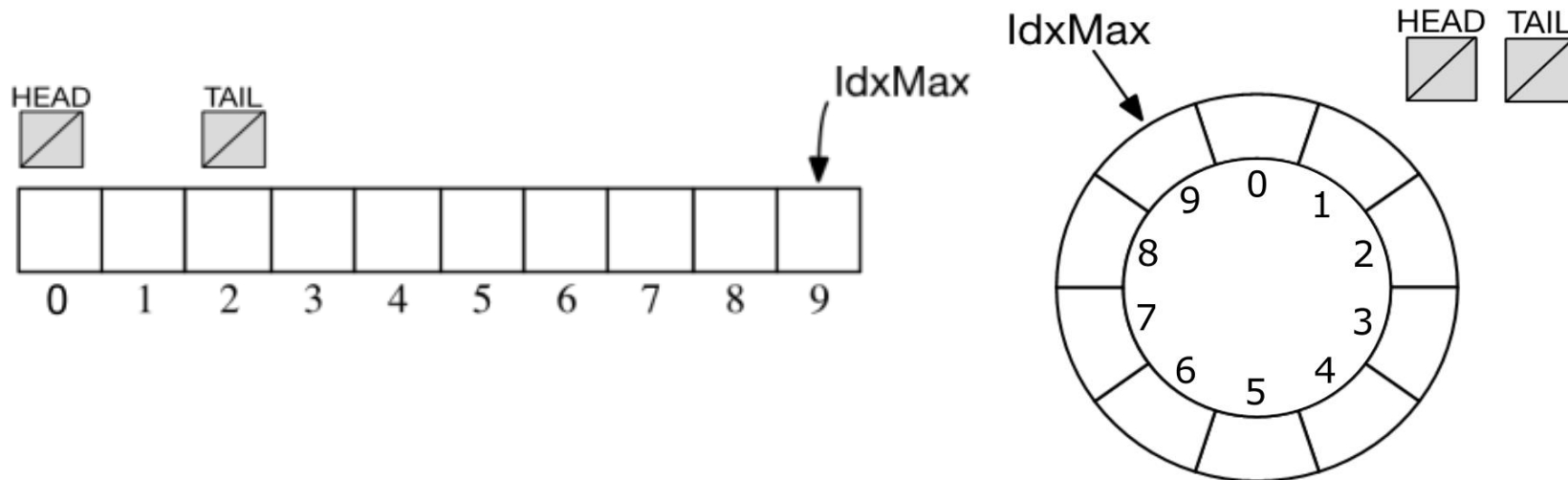
Pergeseran hanya dilakukan jika dan hanya jika $\text{idxTail} = \text{IdxMax}$,
i.e., $\text{idxTail} = \text{CAPACITY} - 1$.

Implementasi Queue dengan array – alt-3

Tabel dengan representasi HEAD dan TAIL yang “berputar” mengelilingi indeks tabel dari awal sampai akhir, kemudian kembali ke awal.

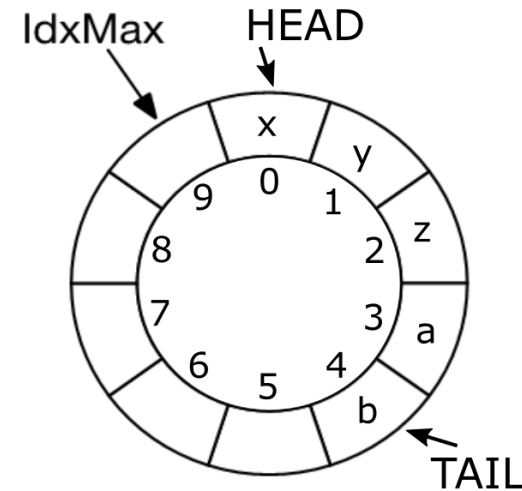
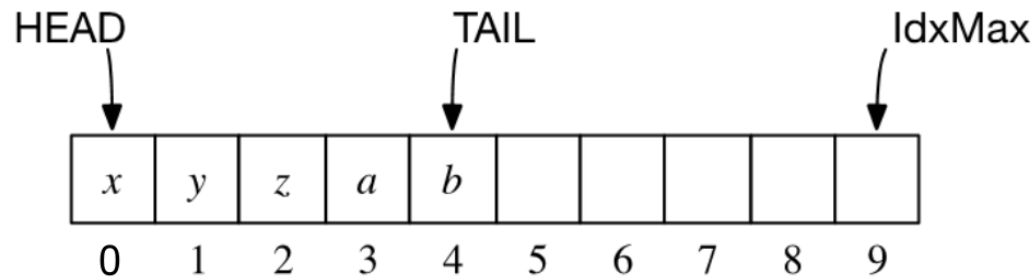
Jika Queue kosong, maka `idxHead` dan `idxTail` = `IDX_UNDEF`.

Representasi ini memungkinkan tidak perlu lagi ada pergeseran yang harus dilakukan seperti pada alt-1 dan alt-2.



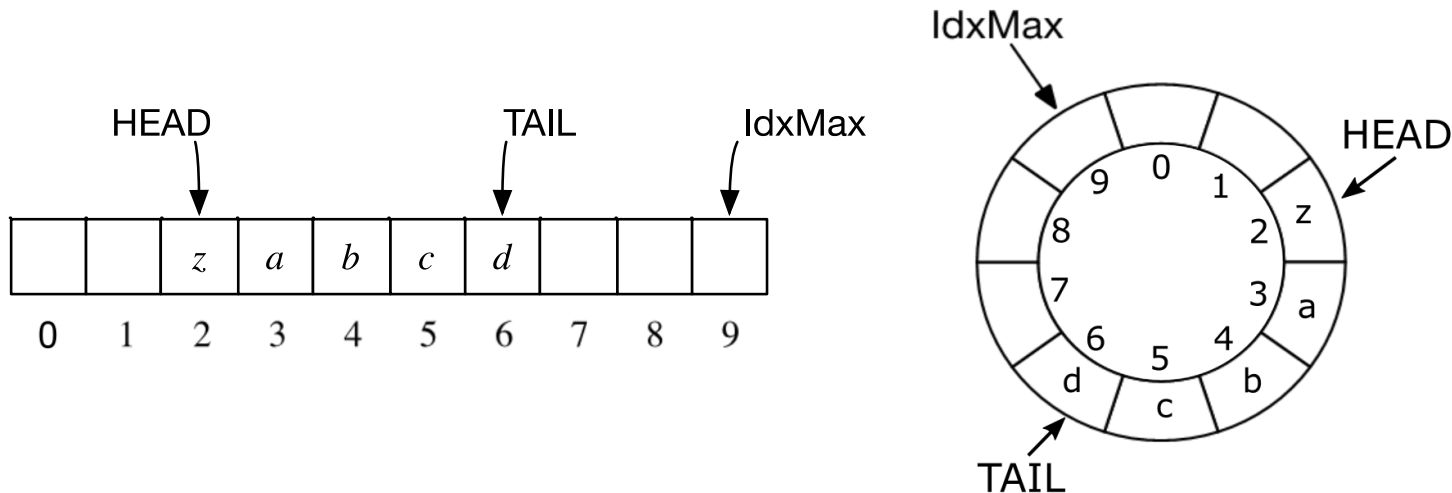
Implementasi Queue dengan array – alt-3

Ilustrasi Queue tidak kosong, dengan 5 elemen, dengan HEAD sedang berada di indeks 0:



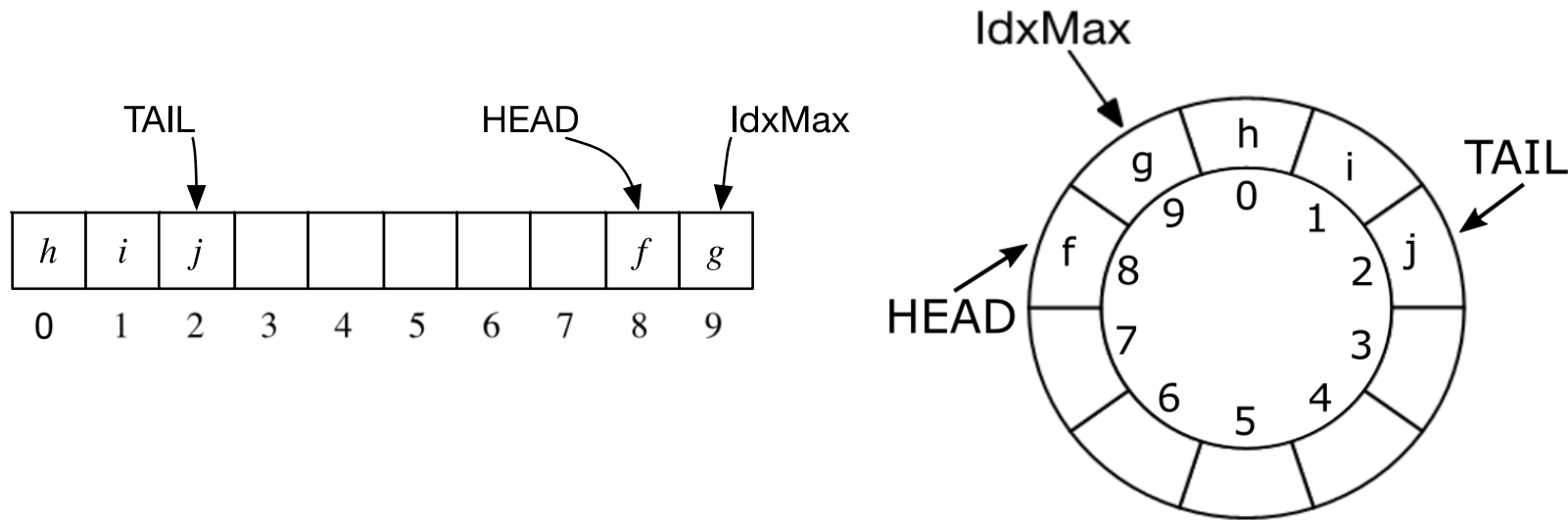
Implementasi Queue dengan array – alt-3

Ilustrasi Queue tidak kosong, dengan 5 elemen, dengan HEAD tidak berada di indeks 0, tetapi **masih “lebih kecil” atau “sebelum” TAIL** (akibat penghapusan/ penambahan):



Implementasi Queue dengan array – alt-3

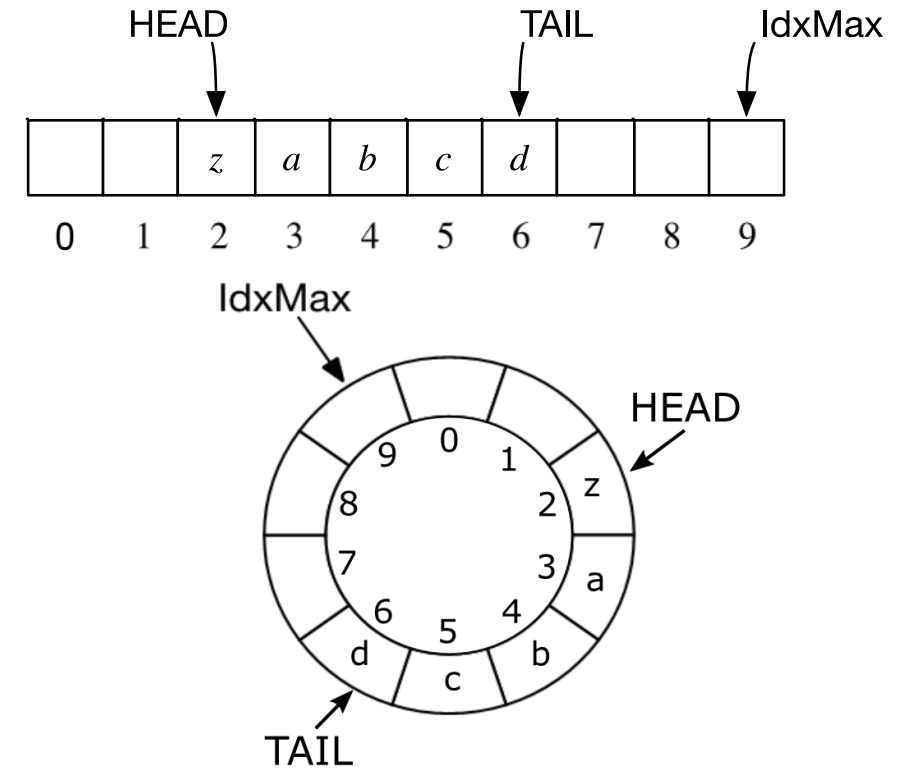
Ilustrasi Queue tidak kosong, dengan 5 elemen, HEAD tidak berada di indeks 0, dan **“lebih besar” atau “sesudah” TAIL** (akibat penghapusan/penambahan):



Implementasi Queue dengan array – alt-3

Algoritma **penambahan elemen**:

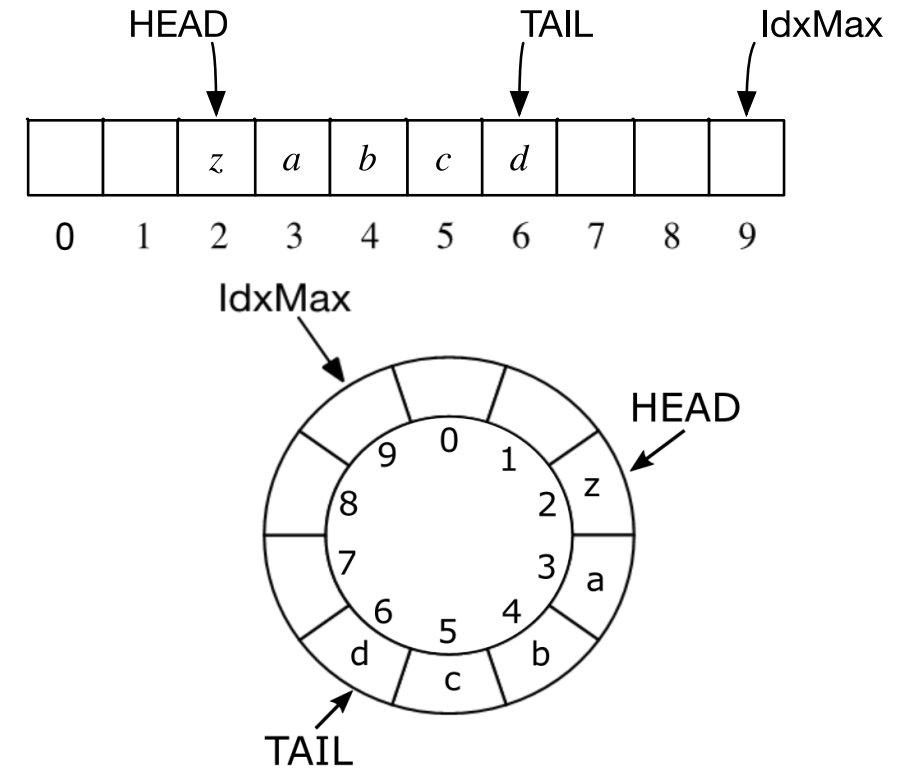
- Jika masih ada tempat: geser TAIL
 - Jika $\text{idxTail} < \text{IdxMax}$: algoritma penambahan elemen sama dengan alt-1 dan alt-2.
 - Jika $\text{idxTail} = \text{IdxMax}$: suksesor dari IdxMax adalah 0 sehingga idxTail yang baru adalah 0.
- **Kasus khusus (Queue kosong)**: idxHead dan idxTail diset = 0.



Implementasi Queue dengan array – alt-3

Algoritma **penghapusan elemen**:

- **Jika Queue tidak kosong:**
 - Ambil nilai elemen HEAD, kemudian HEAD digeser ke kanan.
 - **Jika $\text{idxHead} = \text{IdxMax}$:** idxHead yang baru adalah 0.
- **Kasus khusus** (Queue berelemen 1): idxHead dan idxTail diset = IDX_UNDEF .



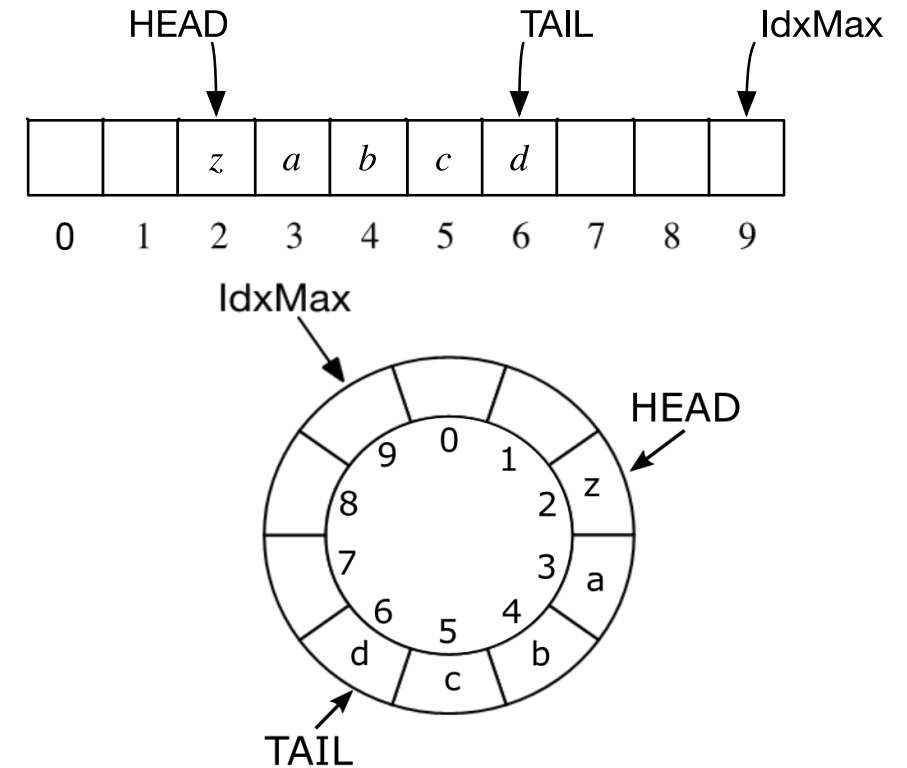
Implementasi Queue dengan array – alt-3

Algoritma ini **efisien** karena tidak perlu pergeseran.

Seringkali strategi pemakaian tabel semacam ini disebut sebagai **circular buffer**.

Salah satu variasi dari representasi pada alt-3:

Menggantikan representasi TAIL dari “**idxTail**” menjadi “**count**” (banyaknya elemen Queue).



Thought exercise

Contoh-contoh sebelumnya menggunakan buffer yang terbatas dan statis. Di sini isFull menjadi relevan meskipun tidak ada di bagian “definisi operasi”.

Renungkan apa yang perlu diubah untuk membuat:

- buffer menjadi dinamis ($q1, q2$: Queue; $q1$ dan $q2$ bisa memiliki kapasitas yang berbeda)
- queue tidak boleh memiliki batas length (length bisa ∞ secara teoretis)

Apa konsekuensinya terhadap model alt-1, alt-2, dan alt-3?

Queue dalam Bahasa C

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

ADT Queue dengan C – alt-2 (alokasi memori statik)

```
/* File: queue.h */
#ifndef QUEUE_H
#define QUEUE_H

#include "boolean.h"
#include <stdlib.h>

#define IDX_UNDEF -1

/* Definisi elemen dan address */
typedef int ElType;
/* Contoh struktur type Queue: array statik, indeks head dan indeks tail disimpan */
typedef struct {
    ElType buffer[CAPACITY];
    int idxHead;
    int idxTail;
} Queue;
/* Definisi Queue kosong: idxHead = idxTail = IDX_UNDEF. */
```

NOTE: if you want flexibility, buffer should be an ElType* and you will need malloc() in CreateQueue. You will also have to have a destructor, e.g., DestroyQueue(), where you call free(buffer).

ADT Queue dengan C – alt-2 (alokasi memori statik)

```
/****** AKSES (Selektor) *****/
#define IDX_HEAD(q) (q).idxHead
#define IDX_TAIL(q) (q).idxTail
#define HEAD(q) (q).buffer[(q).idxHead]
#define TAIL(q) (q).buffer[(q).idxTail]

/****** Konstruktor *****/
void CreateQueue(Queue *q);
/* I.S. Sembarang
   F.S. Membuat sebuah Queue q yang kosong berkapasitas CAPACITY
        jadi indeksnya antara 0..CAPACITY-1
        Ciri Queue kosong: idxHead dan idxTail bernilai IDX_UNDEF */
```

ADT Queue dengan C – alt-2

```
/****** Operasi: pemeriksaan status Queue *****/  
/* catatan: fungsi head(q: Queue) diimplementasikan sebagai macro di halaman  
    sebelumnya */  
  
boolean isEmpty(Queue q);  
/* Mengirim true jika q kosong: lihat definisi di atas */  
  
boolean isFull(Queue q);  
/* Mengirim true jika penyimpanan q penuh */  
  
int length(Queue q);  
/* Mengirim jumlah elemen q saat ini */
```

ADT Queue dengan C – alt-2

```
/** Primitif Add/Delete */  
void enqueue (Queue *q, ElType val);  
/* Proses: Menambahkan val sebagai elemen Queue q.  
   I.S. queue mungkin kosong, TIDAK penuh */  
   F.S. queue bertambah elemen val sebagai tail yang baru, TAIL bergeser ke kanan */  
   Jika IDX_TAIL(queue)=CAPACITY-1, maka geser isi tabel, shg IDX_HEAD(queue)=0 */  
  
void dequeue(Queue *q, ElType* val);  
/* Menghapus head dari Queue q.  
   I.S. queue tidak kosong  
   F.S. val berisi nilai head yang lama.  
       Jika queue tidak menjadi kosong,  
           queue.idxHead berpindah ke elemen berikutnya pada queue.  
       Jika queue menjadi kosong,  
           queue.idxHead dan queue.idxTail menjadi bernilai IDX_UNDEF. */  
  
#endif
```

ADT Queue dengan C – alt-2

```
void CreateQueue(Queue *q) {  
    /* I.S. ... F.S. ... */  
    /* KAMUS LOKAL */  
    /* ALGORITMA */  
    IDX_HEAD(*q) = IDX_UNDEF;  
    IDX_TAIL(*q) = IDX_UNDEF;  
}  
  
boolean isEmpty(Queue q) {  
    /* Mengirim ... */  
    /* KAMUS LOKAL */  
    /* ALGORITMA */  
    return (IDX_HEAD(q) == IDX_UNDEF) && (IDX_TAIL(q) == IDX_UNDEF);  
}
```

ADT Queue dengan C – alt-2

```
boolean isFull(Queue q) {  
    /* Mengirim ... */  
    /* KAMUS LOKAL */  
    /* ALGORITMA */  
    return (IDX_HEAD(q) == 0) && (IDX_TAIL(q) == CAPACITY-1);  
}  
  
int length(Queue q) {  
    /* Mengirim ... */  
    /* KAMUS LOKAL */  
    /* ALGORITMA */  
    return (IDX_TAIL(q) - IDX_HEAD(q)) + 1;  
}
```


ADT Queue dengan C – alt-2

```
void enqueue(Queue *q, ElType val) {
/* I.S. ... F.S. ... */
/* KAMUS LOKAL */
/* ALGORITMA */
    if (isEmpty(*q)) {
        IDX_HEAD(*q) = 0;
        IDX_TAIL(*q) = 0;
    } else { // *q is not empty
        if (IDX_TAIL(*q)==(CAPACITY-1)) { // elemen mentok kanan, geser dulu
            for (int i=IDX_HEAD(*q); i<=IDX_TAIL(*q); i++) {
                (*q).buffer[i-IDX_HEAD(*q)] = (*q).buffer[i];
            }
            IDX_TAIL(*q) -= IDX_HEAD(*q);
            IDX_HEAD(*q) = 0;
        }
        IDX_TAIL(*q)++;
    }
    TAIL(*q) = val;
}
```

ADT Queue dengan C – alt-2

```
void dequeue(Queue *q, ElType *val) {  
    /* I.S. ... F.S. ... */  
    /* KAMUS LOKAL */  
    /* ALGORITMA */  
    *val = HEAD(*q);  
    if (IDX_HEAD(*q) == IDX_TAIL(*q)) {  
        IDX_HEAD(*q) = IDX_UNDEF;  
        IDX_TAIL(*q) = IDX_UNDEF;  
    } else {  
        IDX_HEAD(*q)++;  
    }  
}
```

Latihan Soal: ADT queue

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Soal 1 – Circular Buffer

- a. Definiskan struktur data yang merepresentasi queue bertipe `ElType` yang terdiri atas `<id: integer, cost: integer>` dalam bentuk *circular buffer*, dengan alokasi statik maksimum 100 elemen, dan menyimpan informasi indeks head dan count (banyaknya elemen dalam queue)
- b. Buatlah function `isFull`
- c. Buatlah procedure `enqueue`
- d. Buatlah procedure `dequeue`

Soal 1

function isFull (q: queue) → boolean
{mengirim true jika q penuh}

procedure enqueue (input/output q: queue, input val: ElType)
{Proses: menambahkan val pada q sebagai Tail baru}
{IS: q mungkin kosong, q tidak penuh}
{FS: val menjadi Tail baru dengan mekanisme *circular buffer*}

procedure dequeue (input/output q: queue, output val: ElType)
{Proses: menyimpan nilai Head q ke val dan menghapus Head q}
{IS: q tidak kosong}
{FS: val adalah nilai elemen Head, Head “bergerak” dengan mekanisme *circular buffer*. q mungkin kosong}

Soal 2 – Round Robin

Pandanglah queue pada soal nomor 1 sebagai antrian pekerjaan dengan id adalah nomor identifikasi pekerjaan dan “cost” adalah cost waktu penyelesaian pekerjaan (*time cost*).

Dengan memanfaatkan queue pada soal nomor 1, buatlah procedure **roundRobin** yang memproses queue secara Round Robin, yaitu memproses dengan waktu terbatas T :

- Jika elemen pada HEAD memiliki $\text{cost} \leq T$, elemen tersebut dihapus dari queue.
- Jika elemen pada HEAD memiliki $\text{cost} > T$, maka elemen tersebut dihapus dari queue **dan** disisipkan kembali sebagai Tail dengan cost yang berkurang sebesar T .

Soal 2 – Round Robin

```
procedure roundRobin (input/output q: queue, input t: integer)  
{Proses: memproses elemen antrian q secara round robin}  
{IS: q tidak kosong, t adalah waktu yang tersedia untuk memproses setiap elemen}  
{FS: elemen e pada posisi HEAD dihapus dari q.  
    Jika  $\text{cost } e \leq t$  maka ditampilkan “<id> telah selesai diproses”.  
    Jika  $\text{cost } e > t$  maka e disisipkan kembali sebagai tail q  
    dengan cost berkurang sebesar t }
```

Soal 3 – Priority queue

- a. Dengan memodifikasi queue alternatif 2 pada slide materi kuliah, definisikan (algoritmik) struktur data yang merepresentasi queue yang menggambarkan antrian pekerjaan (job shop). Setiap elemen queue bertipe EType yang terdiri atas $\langle id: \text{integer}, cost: \text{integer} \rangle$. id menunjukkan nomor identifikasi unik dari pekerjaan yang dikelola queue, dan elemen queue terurut membesar berdasarkan “cost” waktu memproses pekerjaan.
- b. Buatlah prosedur enqueue
- c. Buatlah prosedur dequeue

Soal 3 – Priority queue

procedure enqueue (input/output q: queue, input val: ElType)

{Proses: menambahkan val sebagai elemen baru di q, dengan memperhatikan lamanya waktu pekerjaan tsb dapat diselesaikan, yaitu pekerjaan yang lebih cost diletakkan lebih akhir. Jika ada 2 pekerjaan yang cost waktunya sama, pekerjaan terakhir yang baru datang disisipkan lebih belakang}

{IS: q mungkin kosong, q tidak penuh}

{FS: val menjadi elemen q yang baru dengan urutan waktu pekerjaan membesar}

procedure dequeue (input/output q: queue, output val: ElType)

{Proses: menyimpan IDX_UNDEFa head q pada val dan menghapus head dari q}

{IS: q tidak kosong}

{FS: elemen pada HEAD dihapus, dan disimpan nilainya pada val}

Stack (Tumpukan)

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Stack

Stack: list linier yang:

- dikenali elemen puncaknya (TOP),
- aturan penyisipan dan penghapusan elemennya tertentu:
 - Penyisipan selalu dilakukan "di atas" TOP,
 - Penghapusan selalu dilakukan pada TOP.

TOP adalah satu-satunya alamat tempat terjadi operasi.

Elemen Stack tersusun secara LIFO (*Last In First Out*).

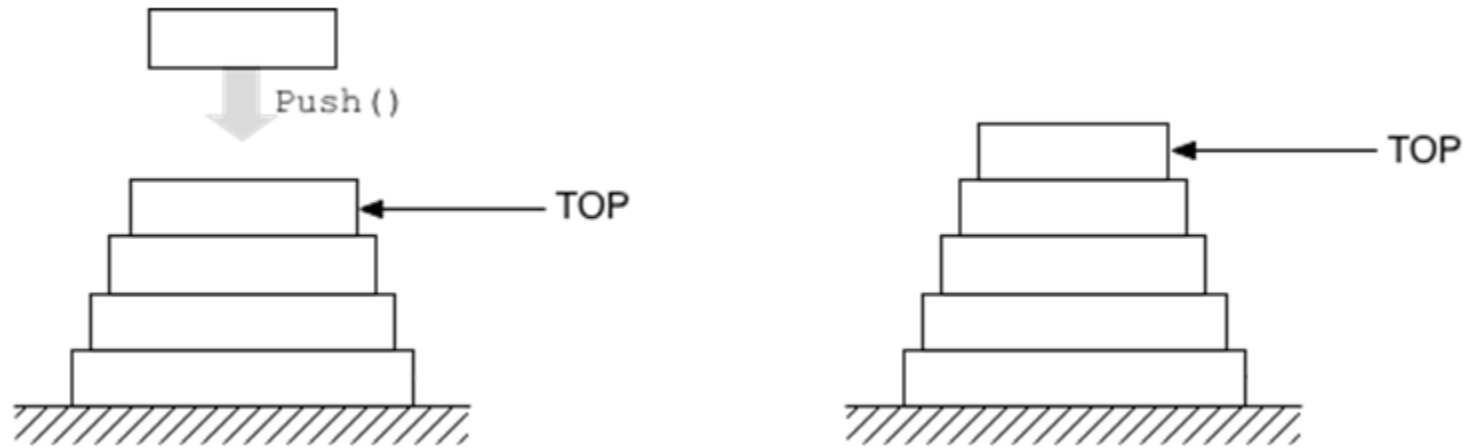


Designed by lifeforstock / Freepik



Lakeside 8206 Two Stack Plate Dispenser

Stack



Stack seperti sebuah List dengan batasan lokasi penambahan & pengurangan elemen.

- Pada **Stack**: operasi penambahan dan pengurangan hanya dilakukan di **salah satu** “ujung” list.
- Pada **List**: operasi **boleh di manapun**.

Tower of Hanoi



https://en.wikipedia.org/wiki/File:Tower_of_Hanoi_4.gif

Pemakaian Stack

- Pemanggilan prosedur
- Perhitungan ekspresi aritmatika
- Rekursivitas
- *Backtracking*

dan algoritma lanjut yang lain

Definisi operasi

Jika diberikan S adalah Stack dengan elemen $ElmtS$

$CreateStack: \rightarrow S$	{ Membuat sebuah tumpukan kosong }
$top: S \rightarrow ElmtS$	{ Mengirimkan elemen teratas S saat ini }
$length: S \rightarrow \underline{integer}$	{ Mengirimkan banyaknya elemen S saat ini }
$push: ElmtS \times S \rightarrow S$	{ Menambahkan sebuah elemen $ElmtS$ sebagai TOP, TOP berubah nilainya }
$pop: S \rightarrow S \times ElmtS$	{ Mengambil nilai elemen TOP, sehingga TOP yang baru adalah elemen yang datang sebelum elemen TOP, mungkin S menjadi kosong }
$isEmpty: S \rightarrow \underline{boolean}$	{ Test stack kosong, true jika S kosong, false jika S tidak kosong }

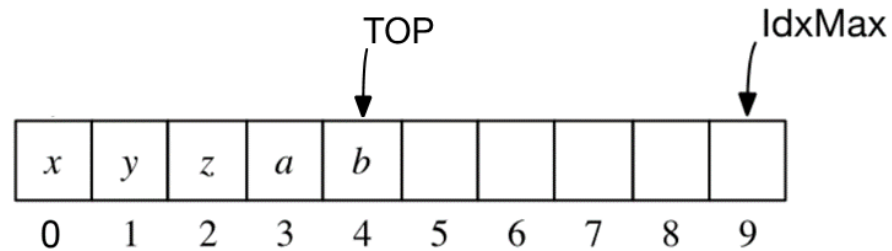
Axiomatic Semantics (fungsional)

- 1) `new()` returns a stack
- 2) `popoff(push(v, S)) = S`
- 3) `top(push(v, S)) = v`

Di mana S adalah Stack dan v adalah value.

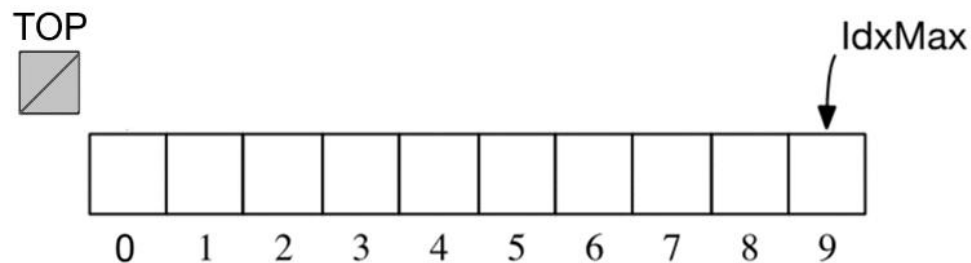
Implementasi Stack dengan List (array)

- Ilustrasi Stack tidak kosong, dengan 5 elemen:



*dengan $\text{IdxMax} = \text{CAPACITY} - 1$

- Ilustrasi Stack kosong, maka TOP diset = IDX_UNDEF.



ADT Stack (dengan array eksplisit-statik)

KAMUS UMUM

constant IDX_UNDEF: integer = -1

constant CAPACITY: integer = 10

type ElType: integer { *elemen Stack* }

{ *Stack dengan array statik* }

type Queue: < buffer: array [0..CAPACITY-1] of ElType, { *penyimpanan elemen* }
idxTop: integer > { *indeks elemen teratas* }

ADT Stack – Konstruktor, akses, & predikat

procedure CreateStack(output s: Stack)

{ *I.S. Sembarang*

*F.S. Membuat sebuah Stack s yang kosong berkapasitas CAPACITY
jadi indeksnya antara 0..CAPACITY-1*

Ciri Queue kosong: idxTop bernilai IDX_UNDEF }

function top(s: Stack) → ElType

{ *Prekondisi: s tidak kosong.*

Mengirim elemen terdepan s, yaitu s.buffer[q.idxTop]. }

function length(s: Stack) → integer

{ *Mengirim jumlah elemen s saat ini }*

function isEmpty(s: Stack) → boolean

{ *Mengirim true jika s kosong: lihat definisi di atas }*

function isFull(s: Stack) → boolean

{ *Mengirim true jika penyimpanan s penuh }*

ADT Stack – Operasi

```
{ *** Menambahkan sebuah elemen ke Stack *** }  
procedure push(input/output s: Stack, input val: ElType)  
{ Menambahkan val sebagai elemen Stack s.  
  I.S. s mungkin kosong, tidak penuh  
  F.S. val menjadi TOP yang baru, TOP bertambah 1 }  
  
{ *** Menghapus sebuah elemen Stack *** }  
procedure pop(input/output s: Stack, output val: ElType)  
{ Menghapus X dari Stack S.  
  I.S. S tidak mungkin kosong  
  F.S. X adalah nilai elemen TOP yang lama, TOP berkurang 1 }
```

ADT Stack (dengan array eksplisit-statik)

procedure CreateStack(output s: Stack)

{ *I.S. Sembarang*

*F.S. Membuat sebuah Stack s yang kosong berkapasitas CAPACITY
jadi indeks nya antara 0..CAPACITY-1*

Ciri stack kosong: idxTop bernilai IDX_UNDEF }

KAMUS LOKAL

-

ALGORITMA

s.idxTop $\leftarrow$ IDX_UNDEF

ADT Stack (dengan array eksplisit-statik)

function isEmpty(s: Stack) → boolean

{ Mengirim true jika Stack kosong: Lihat definisi di atas }

KAMUS LOKAL

-

ALGORITMA

→ s.idxTop = IDX_UNDEF

function isFull(s: Stack) → boolean

{ Mengirim true jika penyimpanan s penuh }

KAMUS LOKAL

-

ALGORITMA

→ s.idxTop = CAPACITY-1

ADT Stack (dengan array eksplisit-statik)

procedure push(input/output s: Stack, input val: EType)
{ *(keterangan tidak ditulis untuk menghemat tempat)* }

KAMUS LOKAL

-

ALGORITMA

s.idxTop $\leftarrow$ s.idxTop + 1
s.buffer[s.idxTop] $\leftarrow$ val

procedure pop(input/output s: Stack, output val: EType)
{ *(keterangan tidak ditulis untuk menghemat tempat)* }

KAMUS LOKAL

-

ALGORITMA

val $\leftarrow$ top(s)
s.idxTop $\leftarrow$ s.idxTop - 1

Thought exercise

Sama seperti pada Queue, contoh-contoh sebelumnya menggunakan buffer yang terbatas dan statis.

Renungkan perubahan seperti pada kasus Queue:

- buffer menjadi dinamis ($s1, s2$: Stack; $s1$ dan $s2$ bisa memiliki kapasitas yang berbeda)
- stack tidak boleh memiliki batas length (length bisa ∞ secara teoretis)

Apa konsekuensinya terhadap implementasi?

Apa bedanya (jika ada) dengan konsekuensi di ADT Queue?

ADT Stack dalam Bahasa C

ADT Stack dalam Bahasa C (dengan array eksplisit-statik)

```
#ifndef STACK_H
#define STACK_H

#include "boolean.h"

#define IDX_UNDEF -1
#define CAPACITY 10

typedef int ElType;
typedef struct {
    ElType buffer[CAPACITY];
    int idxTop;
} Stack;

#define IDX_TOP(s) (s).idxTop
#define TOP(s) (s).buffer[(s).idxTop]
```

ADT Stack dalam Bahasa C (dengan array eksplisit-statik)

```
/** Kontruktor/Kreator */
void CreateStack(Stack *s);
/* I.S. Sembarang */
/* F.S. Membuat sebuah Stack s yang kosong berkapasitas CAPACITY */
/* jadi indeksanya antara 0..CAPACITY-1 */
/* Ciri stack kosong: idxTop bernilai IDX_UNDEF */

/***** Predikat Untuk test keadaan KOLEKSI *****/
boolean isEmpty(Stack s);
/* Mengirim true jika Stack kosong: lihat definisi di atas */

boolean isFull(Stack s);
/* Mengirim true jika Stack penuh */

int length(Stack s);
/* Mengirim ukuran Stack s saat ini */
```

ADT Stack dalam Bahasa C (dengan array eksplisit-statik)

```
/****** Menambahkan sebuah elemen ke Stack *****/  
void push(Stack *s, ElType val);  
/* Menambahkan val sebagai elemen Stack s.  
   I.S. s mungkin kosong, tidak penuh  
   F.S. val menjadi TOP yang baru, TOP bertambah 1 */  
  
/****** Menghapus sebuah elemen Stack *****/  
void pop(Stack *s, ElType *val);  
/* Menghapus X dari Stack S.  
   I.S. S tidak mungkin kosong  
   F.S. X adalah nilai elemen TOP yang lama, TOP berkurang 1 */  
  
#endif
```

ADT Stack dalam Bahasa C (dengan array eksplisit-statik)

```
void CreateStack(Stack *s) {  
    /* ... */  
    /* KAMUS LOKAL */  
    /* ALGORITMA */  
    IDX_TOP(*s) = IDX_UNDEF;  
}
```

```
int length(Stack s) {  
    /* ... */  
    /* KAMUS LOKAL */  
    /* ALGORITMA */  
    return (IDX_TOP(s) + 1);  
}
```

```
boolean isEmpty(Stack s) {  
    /* ... */  
    /* KAMUS LOKAL */  
    /* ALGORITMA */  
    return (IDX_TOP(s) == IDX_UNDEF);  
}
```

```
boolean isFull(Stack s) {  
    /* ... */  
    /* KAMUS LOKAL */  
    /* ALGORITMA */  
    return (IDX_TOP(s) == CAPACITY-1);  
}
```

ADT Stack dalam Bahasa C (dengan array eksplisit-statik)

```
void push(Stack *s, ElType val) {  
    /* ... */  
    /* KAMUS LOKAL */  
    /* ALGORITMA */  
    IDX_TOP(*s)++;  
    TOP(*s) = val;  
}
```

```
void pop(Stack *s, ElType *val) {  
    /* ... */  
    /* KAMUS LOKAL */  
    /* ALGORITMA */  
    *val = TOP(*s);  
    IDX_TOP(*s)--;  
}
```

Contoh Aplikasi ADT Stack

Evaluasi Ekspresi Aritmatika

Evaluasi ekspresi aritmatika yang ditulis dengan *Reverse Polish Notation* (postfix)

Diberikan sebuah ekspresi aritmatika postfix dengan operator ['*', '/', '+', '-', '^']

Operator mempunyai prioritas (prioritas makin besar, artinya makin tinggi)

Operator	Arti	Prioritas
^	pangkat	3
* /	kali, bagi	2
+ -	tambah, kurang	1

Evaluasi Ekspresi Aritmatika

Contoh:

Ekspresi postfix	Arti (ekspresi infix)
$A B * C /$	$(A*B)/C$
$A B C ^ / D E * + A C * -$	$(A/(B^C))+(D*E)-(A*C)$

Digunakan istilah token yaitu satuan “kata” yang mewakili sebuah operan (konstanta atau nama) atau sebuah operator.

Mesin Evaluasi Ekspresi

Program EKSPRESI

{ Menghitung sebuah ekspresi aritmatika yang ditulis secara postfix }

USE STACK { paket stack sudah terdefinisi dgn elemennya bertipe token }

KAMUS

type Token: ... { terdefinisi }
s: Stack { stack of token }
currentToken, op1, op2: Token { token: operand U operator }

procedure firstToken

{ Mengambil token yang pertama, disimpan di currentToken }

procedure nextToken

{ Mengambil token yang berikutnya, disimpan di currentToken }

function endToken → boolean

{ Menghasilkan true jika proses akuisisi mendapat hasil sebuah token kosong.
Merupakan akhir ekspresi. }

function isOperator → boolean

{ Menghasilkan true jika currentToken adalah operator }

function evaluate(op1,op2,operator: token) → token

{ Menghitung ekspresi, mengkonversi hasil menjadi token }

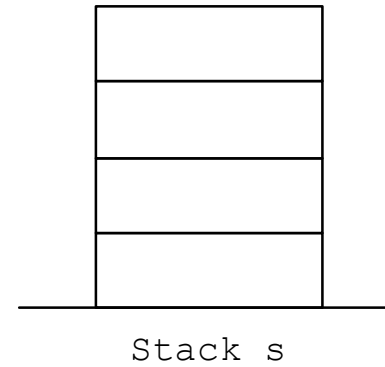
(lanjutan)

ALGORITMA

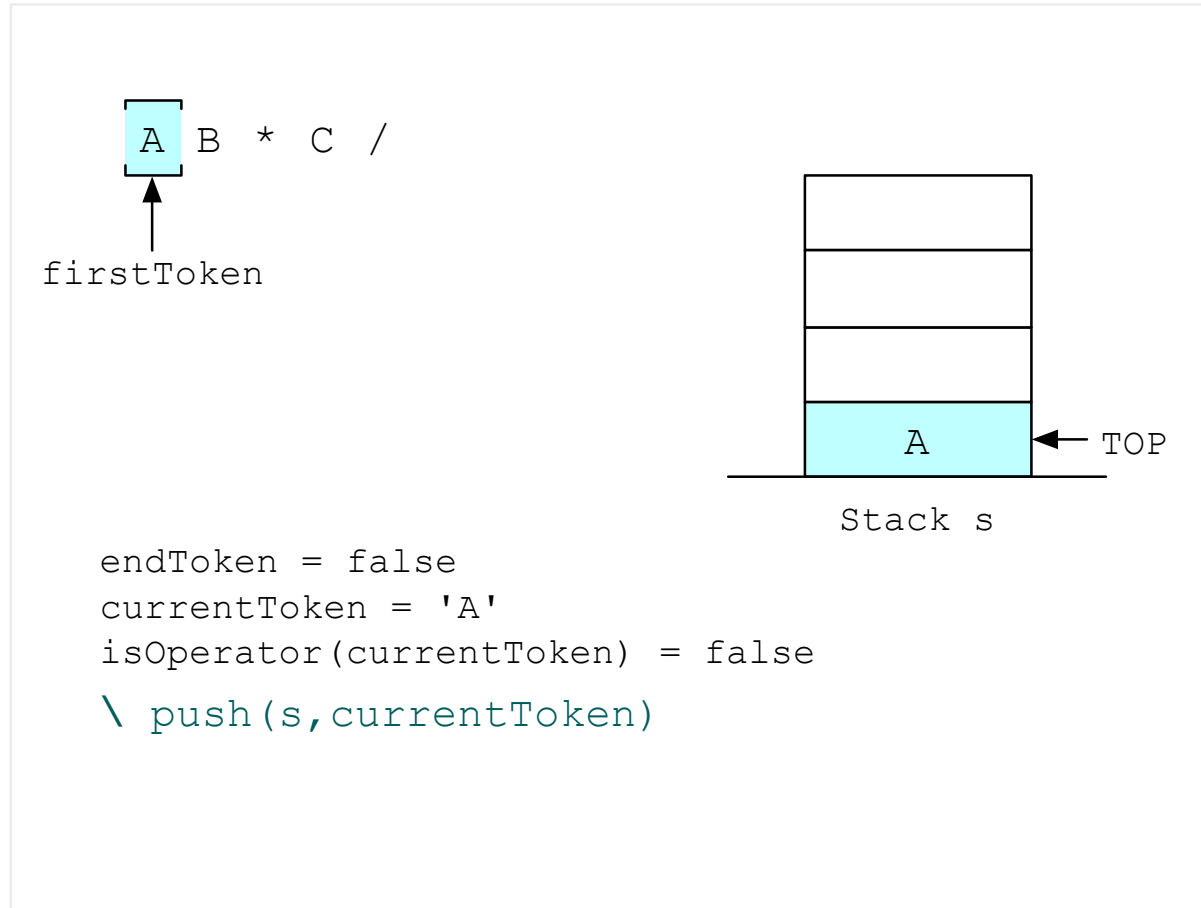
```
firstToken
if (endToken) then
    output ("Ekspresi kosong")
else
    repeat
        if not isOperator then
            push(s,currentToken)
        else
            pop(s,op2)
            pop(s,op1)
            push(s,evaluate(op1,op2, currentToken))
    nextToken
until (endToken)
output (top(s)) { Menuliskan hasil }
```

Evaluasi Ekspresi Aritmatika

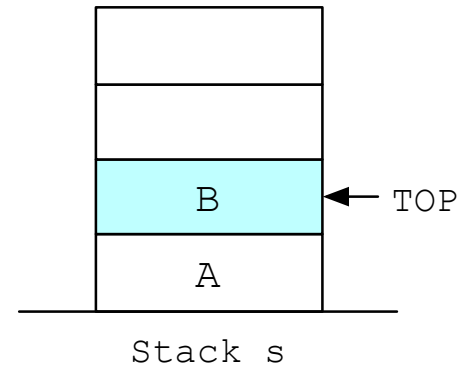
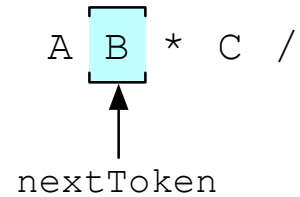
A B * C /



Evaluasi Ekspresi Aritmatika

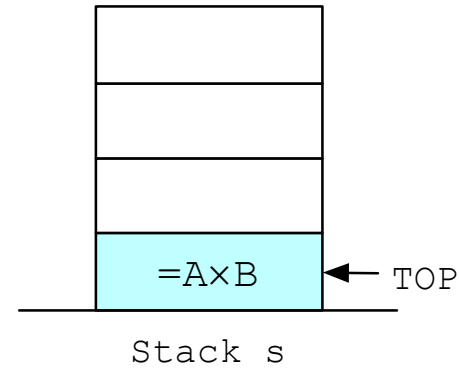
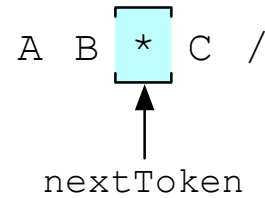


Evaluasi Ekspresi Aritmatika



```
endToken = false  
currentToken = 'B'  
isOperator(currentToken) = false  
\ push(s, currentToken)
```

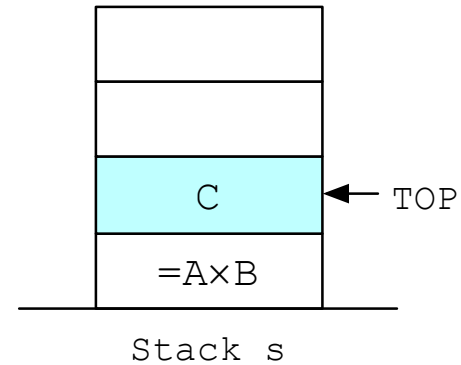
Evaluasi Ekspresi Aritmatika



```
endToken = false
currentToken = '*'
isOperator(currentToken) = true
\ pop(s, op2)
  pop(s, op1)
  push(s, evaluate(op1, op2, currentToken))
```


Evaluasi Ekspresi Aritmatika

A B * C /
 ↑
 nextToken

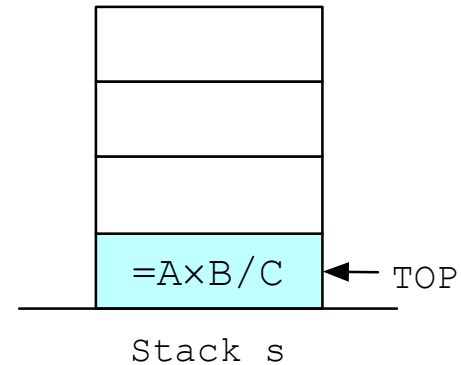


```
endToken = false  
currentToken = 'C'  
isOperator(currentToken) = false  
\  push(s, currentToken)
```

Evaluasi Ekspresi Aritmatika

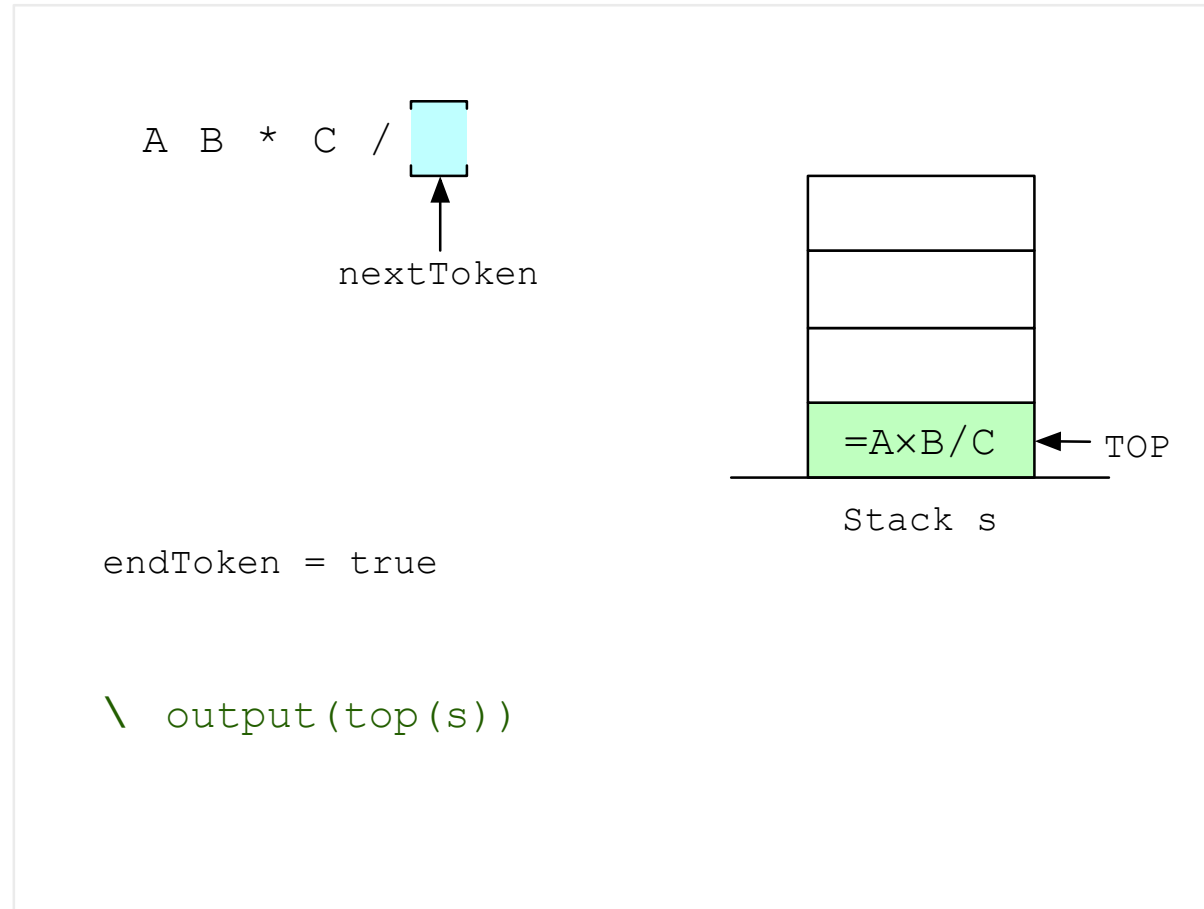
A B * C /

↑
nextToken

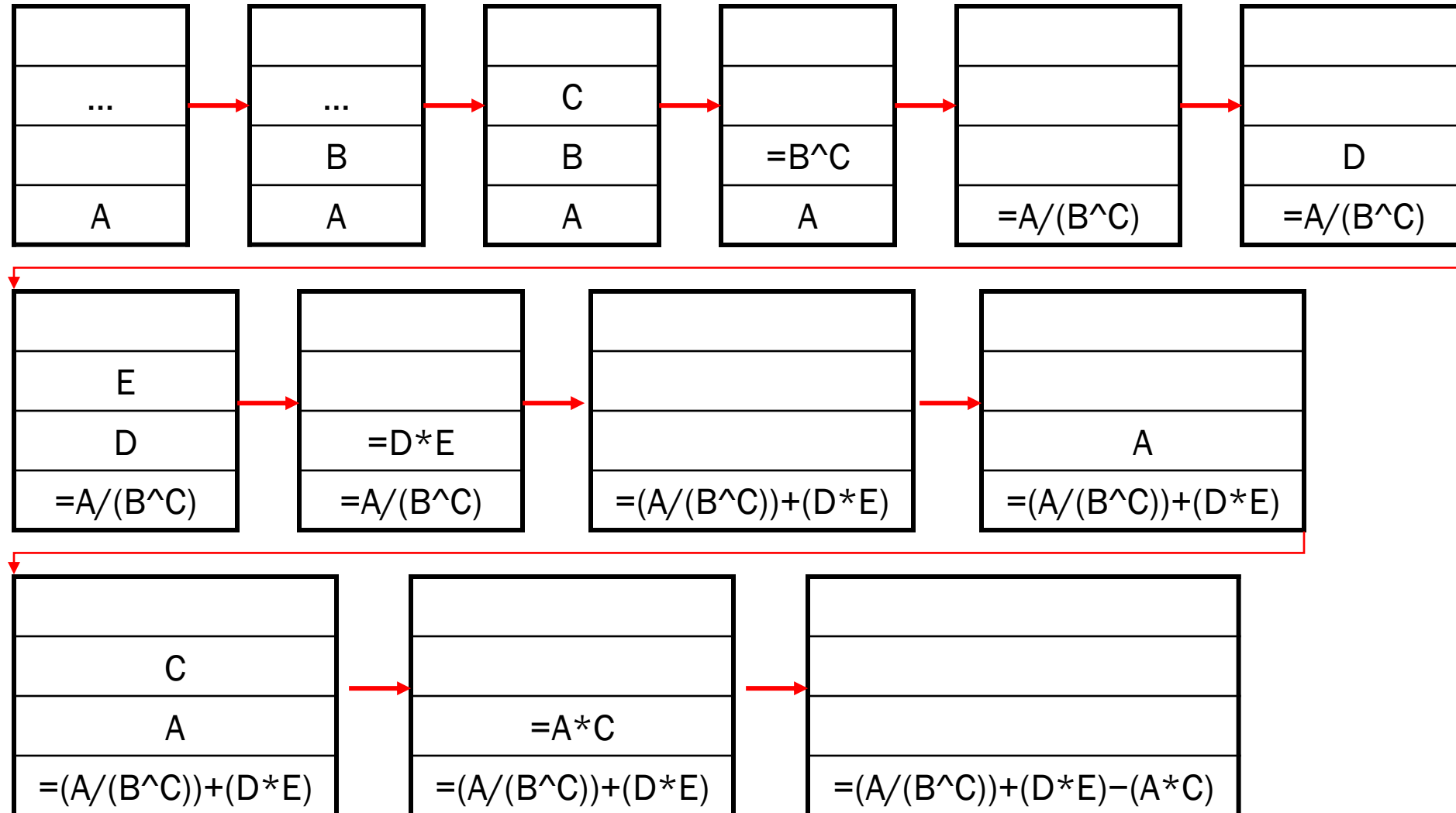


```
endToken = false
currentToken = '/'
isOperator(currentToken) = true
\ pop(s,op2)
  pop(s,op1)
  push(s,evaluate(op1,op2,currentToken))
```

Evaluasi Ekspresi Aritmatika



$ABC^{\wedge}/DE^{*}+AC^{*}-$



Latihan Soal: ADT Stack

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Soal 1

Dengan menggunakan ADT Stack yang direpresentasikan sebagai array statik-eksplisit seperti yang dibahas di materi kuliah, realisasikan prosedur dan fungsi berikut ini:

```
procedure copyStack(input sIn: Stack, output sOut: Stack)
{ Membuat salinan sOut }
{ I.S. sIn terdefinisi, sOut sembarang }
{ F.S. sOut berisi salinan sIn yang identik }
```

Soal 1

procedure inverseStack(input/output s: Stack)

{ Membalik isi suatu stack }

{ I.S. s terdefinisi }

{ F.S. Isi s terbalik dari posisi semula }

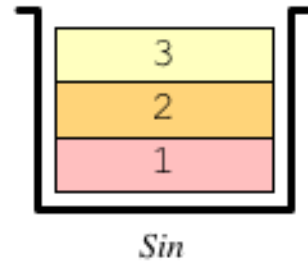
function mergeStack(s1,s2: Stack) → Stack

{ Menghasilkan sebuah stack yang merupakan hasil penggabungan s1 dengan s2 dengan s1 berada di posisi lebih “bawah”. Urutan kedua stack harus sama seperti aslinya. }

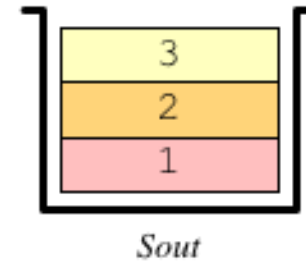
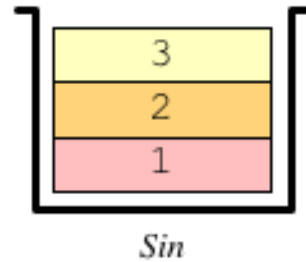
{ Stack baru diisi sampai seluruh elemen s1 dan s2 masuk ke dalam stack, atau jika stack baru sudah penuh, maka elemen yang dimasukkan adalah secukupnya yang dapat ditampung. }

CopyStack(*Sin*, *Sout*)

Initial State:

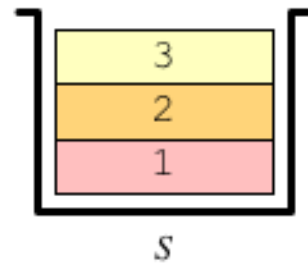


Final State:

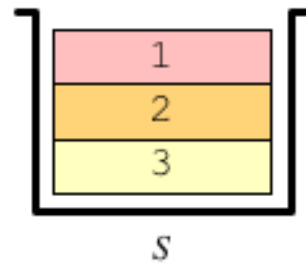


InverseStack(*S*)

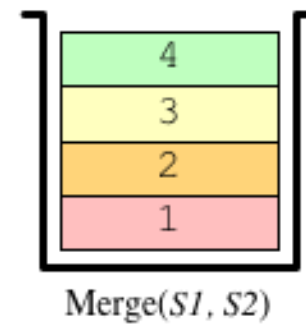
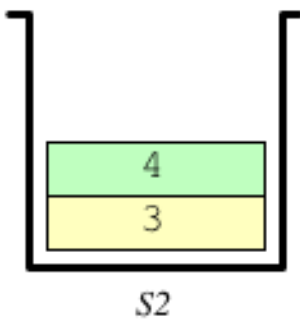
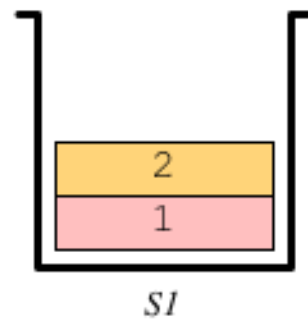
Initial State:



Final State:



MergeStack(*S1*, *S2*)



Soal 2

Dengan memanfaatkan mesin kata (versi 1), modifikasi dan lengkapi program Evaluasi Ekspresi Aritmatika pada slide materi kuliah.

Pita karakter berisi ekspresi aritmatika dengan masing-masing “kata” dipisahkan oleh satu atau lebih BLANK.

Kata yang merupakan operan merepresentasikan sebuah bilangan, contoh: “123” merepresentasikan bilangan 123

Contoh isi pita: 123 3 *

Dengan demikian, harus dibuat ADT Stack of Kata. Gunakan ADT Stack dengan representasi array statik-eksplisit.

Harus dibuat pula fungsi yang mengubah suatu Kata yang merepresentasikan operan menjadi angka/integer. Diasumsikan hanya ada integer positif atau 0.